

Hacia el Aprendizaje de Máquina Cuántico para el Análisis de Código Malicioso [13]

Explicación teórica y recreación
práctica de los experimentos

Autor

Santiago Rocha Pachón
srochap@unal.edu.co

Afiliación

Universidad Nacional de Colombia / Quantum Programming

Fecha

18 de junio de 2026

Índice general

1. Introducción	3
1.1. Motivación	3
1.2. Terminología	4
1.3. Bases de datos usadas	5
1.3.1. AndroZoo (AZ) [1]	5
1.3.2. EMBER [2]	6
1.3.3. API-Graph [28]	6
2. Antecedentes	8
2.1. En la detección de <i>malware</i>	8
2.2. En otros dominios	9
3. QMLP	10
3.1. Arquitectura	10
3.1.1. Circuitos Cuánticos Variacionales (VQCs)	10
3.1.2. Compuertas de capa o capas de procesamiento	11
3.1.3. Un QMLP estándar	12
3.1.4. Usada en el artículo	13
3.2. Implementación	14
4. QCNN	16
4.1. Arquitectura	16
4.1.1. Usada en el artículo	17
4.2. Implementación	19
5. Implementación	21
5.1. Herramientas usadas	21
5.2. Modificaciones propuestas	22

6. Recreación de los experimentos	23
7. Discusión	26
8. Bibliografía	28
9. Anexos	31

1 Introducción

En “*Towards Quantum Machine Learning for Malicious Code Analysis*” [13], referido de ahora en adelante como ‘el artículo’, se exploran, mediante simulaciones en computadores clásicos, los desempeños de dos redes neuronales cuánticas híbridas en materia de clasificación, multiclase y binaria, de aplicaciones móviles y archivos ejecutables en el contexto de la clasificación e identificación de archivos maliciosos o *malware*.

Dichos modelos son, puntualmente, un perceptrón cuántico multicapa (QMLP por sus siglas en inglés) y una red neuronal convolucional cuántica (QCNN también por sus siglas en inglés), ambas entrenadas usando una porción significativa (decenas a cientos de miles de entradas) de las bases de datos API-Graph [28], AndroZoo [1] y EMBER [2].

El objetivo del presente trabajo es replicar y ahondar en los detalles de los experimentos realizados en el artículo, con el fin de comprender cómo se están implementando, entrenando y evaluando las QCNNs híbridas y los QMLPs híbridos en la práctica ^a, qué tan ‘de libre acceso’ son las bases de datos utilizadas y, en particular, qué tan replicable es este trabajo como primer acercamiento a la investigación del autor.

Conceptos clave: QML, QCNN, QMLP, identificación y clasificación de *malware*, replicación.

^aDe ahora en adelante, en todos los capítulos salvo en el capítulo 2, se sobreentenderá que al mencionar una QCNN o a un QMLP, se tratará de dicho modelo en su formato híbrido.

1.1 Motivación

La tarea de identificar si un archivo es malicioso (*malware*) o no, y si lo es, a qué familia pertenece, requiere de múltiples análisis que no se basan en reglas condicionales. Es decir, teniendo toda la información de un análisis estático (capturando el código del archivo) y dinámico (ejecutando el archivo), no siempre es posible determinar si el archivo en cuestión es malicioso o no. Esto se debe, por un lado, a que los creadores de *malware* pueden usar técnicas como la ofuscación para enmascarar el código de los programas sin que su propósito difiera de lo inicialmente planteado [27] [14]; y por otro, a que algunos programas son capaces de identificar cuándo están siendo ejecutados para su análisis y usan

herramientas anti-análisis para confundir a los investigadores o detectores de *malware* [11].

Adicionalmente, el análisis dinámico en particular requiere de recursos de cómputo más demandantes que no garantizan una certeza total en los análisis ejecutados con ellos.

Por otra parte, la información que se obtiene de un análisis estático puede llegar a expresarse en más de un millón de *features* o características [17] y las bases de datos a trabajar pueden llegar a tener cientos de miles de entradas únicas (ver 1.3).

Por otro lado, como se describirá en profundidad en 2, las estrategias de QML usadas en diferentes dominios del conocimiento tienen buenos rendimientos cuando la entrada se propone como una imagen. Por ejemplo, la ofuscación de una aplicación puede verse como ruido estático en una imagen. Asimismo, las implementaciones cuánticas de los modelos a tratar reducen la complejidad computacional requerida para entrenar y hacer inferencia con estos modelos [18] [6] [9].

Por todo lo anterior, resulta pertinente explorar qué tan buenas son estrategias de QML para clasificar *malware* (de manera binaria y multiclase) a partir de datos de análisis estáticos representados como imágenes. Además, resulta natural la elección de una QCNN para esta tarea [4], mientras que implementar un QMLP tiene una motivación exploratoria y de todas manera válida.

Por último, los autores publicaron el código con el que realizaron sus experimentos, por lo que es posible realizar un análisis tanto práctico como teórico en el tema tratado por ellos (y que compagina intereses académicos principales del autor de este trabajo).

1.2 Terminología

Ya sea porque el lector no esté tan familiarizado con los conceptos a tratar, o por garantizar claridad, téngase en cuenta que:

- El **malware** se define como “cualquier programa informático que tiene efectos no deseados o maliciosos”¹.
- Un **archivo ejecutable portable** (PE por sus siglas en inglés) es cualquier archivo que contiene una imagen (ejecutable) u objetos, y que está concebido para ejecutarse en el sistema operativo Windows². Una bases de datos de PEs, en el contexto de este trabajo, puede consistir en un registro de los metadatos de varios PEs que se obtienen previo a ejecutarlos (es decir, al realizar un análisis estático) y/o en una serie de características resultantes de un análisis hecho por herramientas como VirusTotal³ (que pueden incluir ejecutar el archivo en un entorno aislado como una máquina virtual).
- Una **aplicación de Android** es un programa de *software* compuesto de múltiples elementos conectados para presentar al usuario una interfaz con una lógica compu-

¹Tomado el 13 de junio de 2026 de <https://www.mintic.gov.co/portal/inicio/18744:Malware>

²Tomado el 13 de junio de 2026 de <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>

³<https://www.virustotal.com/gui/home/upload> consultado el 13 de junio de 2026

tacional de fondo ⁴. Además, dicho programa está pensado para ejecutarse en un dispositivo (usualmente móvil) con la capacidad de ejecutar el sistema operativo Android. Tal como en el caso de los PEs, una base de datos de aplicaciones de Android en el contexto que compete este trabajo puede contener metadatos y/o análisis realizados a las aplicaciones. Sin embargo, a diferencia de las bases de datos de PEs a utilizar, la información puede ser presentada como un grafo, como en el caso de API-Graph [28].

Sobre redes neuronales

Se asume que el lector está familiarizado con las redes neuronales clásicas de alimentación directa (FFNNs por sus siglas en inglés) y en particular, las profundas completamente conectadas (FC por sus siglas en inglés) o perceptrones multicapa (MLP por sus siglas en inglés), y las convolucionales (CNN por sus siglas en inglés).

En caso contrario, se sugiere revisar dichos temas y los necesarios para comprenderlos en [19] y [4].

1.3 Bases de datos usadas

1.3.1 AndroZoo (AZ) [1]

Consiste en una colección de 3,182,590 aplicaciones de Android descargadas de distintos proveedores o mercados (no solo Google Play, que es el principal) y su información asociada, incluyendo los resultados de un análisis de VirusTotal.

Los autores del artículo, sin embargo, no usaron la base de datos entera sino la porción obtenida en [17] ⁵ y en [22] ⁶. Respectivamente, corresponden a la base de datos que en el artículo (y en lo que resta de este trabajo) llaman AZ-Class (hasta 184,342 entradas de entrenamiento y 20,482 de evaluación, con 2,439 *features*), usada en la tarea de clasificar aplicaciones maliciosas registradas en AZ en las 4, 14 y 23 familias más comunes; y a la referida como AZ-Domain (67,656 entradas de entrenamiento y 68,330 de evaluación, con 1,789 *features*).

Vale la pena mencionar que no fue nada fácil encontrar la porción exacta de AZ que se usó en el artículo. Incluso, el autor de este trabajo solicitó (y satisfactoriamente se lo dieron) permiso a una división de la Universidad de Luxemburgo para poder extraer la base de datos, pero ciertamente no es lo ideal. Por lo tanto, se habría apreciado bastante que los autores del artículo hubieran puesto los hipervínculos que llevan a las descargas (sin permiso ni autenticación) de las porciones usadas.

1.3.2 EMBER [2]

⁴Tomado el 13 de junio de 2026 de <https://developer.android.com/topic/architecture>

⁵A fecha de 13 de junio de 2026, se podía descargar de <https://zenodo.org/records/14537891>

⁶A fecha de 13 de junio de 2026, se podía descargar de <https://huggingface.co/datasets/IQSeC-Lab/MADAR>

Acrónimo para *Endgame Malware BEenchmark for Research*, consta de 1.1 millones de PEs (de Windows) dividida en particiones de entrenamiento y etiquetado (cuya distribución varía de acuerdo a la variante). Además, tiene tres variantes: la 2017 versión 1, la 2017 versión 2 y la 2018 versión 2 (más retadora para los clasificadores [21]). El año de la variante corresponde a la fecha más reciente de la que se pudieron obtener los PEs (por ejemplo, hasta el 31 de diciembre de 2017 en el caso de 2017), y la versión corresponde a dos conjuntos de *features* diferentes.

No obstante, tal como con AZ (1.3.1), los autores usaron solo una muestra de EMBER 2018 versión 2, cuya obtención viene de [21]. En dicha publicación, para la tarea de clasificación por familias (*Class-IL*, referida como EMBER-Class de ahora en adelante), los autores recolectaron únicamente la información de los PEs maliciosos, reportados en 2018, pertenecientes a las n familias de *malware* más frecuentes en la base de datos, donde n variaba entre 4, 14 y 23 (usada en la recreación de los experimentos), resultando en 89,281 muestras de entrenamiento y 9,930 de evaluación; 184,770 muestras de entrenamiento y 20,464 de evaluación; y 220,412 muestras de entrenamiento y 24,445 de evaluación, respectivamente, con 2,381 *features* por cada una.

Similarmente, para la tarea de clasificación binaria (*Domain-IL*, referida como EMBER-Domain en lo que resta), usaron todas las entradas etiquetadas (como malignas o benignas) de 2018, eliminando únicamente aquellas que no estuvieran clasificadas. Esto resultó en 141,817 muestras de entrenamiento y 75,006 de evaluación, todas, a su vez, con 2,381 *features*.

Vale la pena resaltar que para obtenerlos, se debe descargar la base de datos entera de EMBER 2018 versión 2 ⁷ junto con dos archivos de Python y/o cuadernos de Jupyter del repositorio de [21] que permiten, con algunas pequeñas modificaciones en el caso del usado para la tarea de clasificación por familias, obtener todas las particiones de la base de datos antes mencionadas ⁸

1.3.3 API-Graph [28]

Esta base de datos contiene información de 322,594 aplicaciones de Android, de las cuales 32,089 (10 %) son maliciosas y 290,505 (90 %) son benignas. El procedimiento para seleccionaras fue el siguiente: se descargaron todos los *malware* disponibles en tres bases de datos abiertos reportados entre el 2012 y el 2018, y se conservaron únicamente aquellos archivos que fueran clasificados como *malware* por 15 o más antivirus tras el análisis de VirusTotal. Similarmente, para obtener la porción de aplicaciones benignas, se descargaron más de un millón de aplicaciones publicadas en Google Play ⁹ de AndroZoo [1] (1.3.1) y se seleccionaron solo aquellas que no fueran clasificadas como *malware* por

⁷A fecha de 15 de junio del 2026, se podía obtener en <https://github.com/elastic/ember>, el repositorio asociado a [2]

⁸Específicamente, los archivos son este cuaderno de Jupyter o su equivalente en Python para la clasificación por familias y este para la clasificación binaria (disponibles a 8 de junio del 2026)

⁹Consultado el 13 de junio de 2026 de <https://play.google.com/store/games>

ningún antivirus tras el análisis de VirusTotal.

Hecho lo anterior, dada una aplicación, se crea un grafo por cada llamado de la aplicación a una API (interfaz de aplicación) cuyos nodos son: métodos, clases, paquetes y/o permisos involucrados en dicho llamado. Posteriormente, los grafos se codifican en un único vector de un número fijo (para todas las aplicaciones) de entradas.

No obstante, solo está disponible la base de datos de todos los identificadores *hash* de cada aplicación, y puesto que las aplicaciones usadas se descargaron de tres bases diferentes y los autores no especificaron cómo obtuvieron la porción que usaron, no fue posible recrear los experimentos con esta base de datos.

2 Antecedentes

Para tenerlo presente, vale la pena mencionar que el paper se basa en los trabajos pioneros de la QCNN [9] y del QMLP [6] para diseñar sus arquitecturas. En suma, muchos de los trabajos en los últimos años ha optado por modelos híbridos clásicos-cuánticos ¹ antes que puramente cuánticos, en especial por la consideración de que cualquiera de estos modelos se utilizaría en computadoras Cuánticas de Escala Intermedia con Ruido (o NISQ por sus siglas en inglés) [23].

2.1 En la detección de *malware*

La primera publicación en trabajar la aplicación de aprendizaje de máquina cuántico en la detección de *malware*, usando una representación del mismo como imagen fue [14].

Posteriormente, trabajando en el análisis de *malware* embebido en aplicaciones de Andorid, se implementó y evaluó una QCNN híbrida para clasificar ciertas aplicaciones (recolectadas de seis bases de datos) como benignas o malignas en [8]; y un modelo muy similar se usó en [24] para clasificar (en familias de *malware*) las aplicaciones de la base de datos Maling ².

Paralelamente, se investigó la aplicación de una QCNN entrenada con archivos que contienen algún tipo (de los 25 considerados) de *malware* (obtenidos de Malevis ³), representados como imágenes (no a blanco y negro, sin embargo), en la categorización de estos archivos [7].

A pesar de que la mayoría de los investigadores usan la representación de los *malware* como imágenes, otros autores encontraron que no todas las representaciones como imágenes han sido efectivas, más sin embargo, modificaron el método *Grayscale* y optimizaron la función de ponderación para las distintas secciones resultantes del método mencionado para mejorar la interpretabilidad de la información provista a los modelos

¹ Los modelos híbridos se categorizan en dos familias: los clásico a cuántico (CQ por sus siglas en inglés), que se caracterizan por usar circuitos clásicos para el procesamiento de las *features* con las que se alimentará a un modelo de aprendizaje de máquina (ML por sus siglas en inglés); y los cuántico a clásico (QC por sus siglas en inglés), en los que se usan circuitos cuánticos para extraer *features* de la información con la que se cuenta que luego se enviará a un modelo de ML implementado con circuitos clásicos [23].

² Tomado el 13 de junio de 2026 de <https://www.kaggle.com/datasets/manmandes/malimg>.

³ Tomado el 13 de junio de 2026 de <https://www.kaggle.com/datasets/sohamkumar1703/malevis-dataset/data>.

[20]. Esto permitió una mejora del 20 % respecto a los experimentos anteriores y una precisión y puntaje $F1$ superiores al 80 %.

Finalmente, en [12] se analizó el desempeño de una Máquina Cuántica de Vectores de Soporte (QSVM por sus siglas en inglés) híbrida en la clasificación de PEs como benignos o maliciosos. Lo más interesante es que documentaron su experiencia implementando este modelo usando Qiskit y probándolo en distintas computadoras cuánticas de IBM.

2.2 En otros dominios

Se remite al lector a [23] para un informe en detalle del progreso en el campo de QML, y se mencionarán a continuación únicamente los trabajos citados en el paper.

Por un lado, se plantearon y evaluaron cuatro arquitecturas de redes neuronales cuánticas (QNNs por sus siglas en inglés) de 4 y 10 qubits, usando una capa o bloque para codificar las entradas, y otro que divergía según la arquitectura elegida pero que se basaba en compuertas de rotación (RX , RY , RZ y $U3$) y/o de control ($CU3$) [26]. Las redes tenían entradas de audio o imágenes y su objetivo era clasificar dichas entradas en dos o cuatro clases. Algo similar se realizó en [25], en el que se exploraron seis diseños diferentes de QNNs y su desempeño en tareas similares a las mencionadas anteriormente, con un énfasis extra en buscar la robustez y tolerancia al ruido.

De manera similar, en [15] se implementaron tres modelos de QNN: uno base, otro eficiente en *hardware* y una QNN profunda. Hecho esto, se evaluó el desempeño de estos clasificando imágenes en dos categorías.

Por último, en [6] se implementó un QMLP que mejoró la exactitud a la hora de clasificar (en 10 categorías) imágenes en un 10 % respecto a trabajos previos con menos compuertas y recursos.

3 QMLP

3.1 Arquitectura

Para entender la arquitectura de un QMLP (y de la QCNN a tratar en 4), se debe recurrir a sus unidades elementales: los circuitos cuánticos variacionales (VQCs por sus siglas en inglés) y su agrupación en compuertas de capa.

3.1.1 Circuitos Cuánticos Variacionales (VQCs)

Es un circuito cuántico parametrizado (de n qubits) que recibe un estado $|\phi_0\rangle$ y aplica una transformación unitaria $U(\theta)$ parametrizada por un conjunto de variables libres θ [6] [5], determinado por un *ansatz*. En otras palabras, transforma un vector de entrada $|\phi_0\rangle$ en un vector de salida $U(\theta)|\phi_0\rangle$.

Un *ansatz* es una operación (que se suele ver como un esquema) que indica qué parámetros forman θ , cómo se usan dentro del circuito (la forma de U), y, por tanto, cómo se pueden optimizar; pues, a fin de cuentas, tal como en el aprendizaje de máquina clásico, el objetivo de una optimización de una red neuronal cuántica (QNN) es encontrar el argumento mínimo θ de alguna función de costo C , es decir, se busca

$$\hat{\theta} := \arg \min_{\theta} C(\theta)$$

A pesar de que un *ansatz* tiene una estructura particular en cada problema, se puede

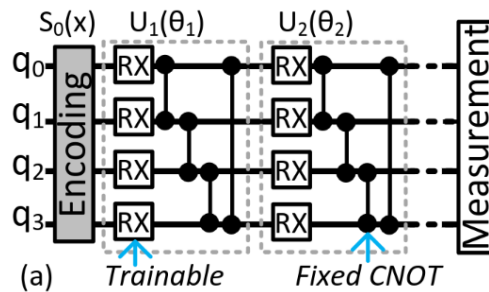


Figura 3.1: Una QNN con dos VQC estándar, tomada de [6].

expresar, genéricamente, como una secuencia finita de transformaciones unitarias parametrizadas así:

$$U(\theta) := U_k(\theta_k) U_{k-1}(\theta_{k-1}) \dots U_2(\theta_2) U_1(\theta_1)$$

donde $\theta = (\theta_k, \theta_{k-1}, \dots, \theta_2, \theta_1)$ (o bien $\theta = \{\theta_k, \theta_{k-1}, \dots, \theta_2, \theta_1\}$) es el conjunto de parámetros (que en redes neuronales clásicas serían los pesos y sesgos de cada capa).

En la figura 3.1, se puede evidenciar el uso de un VQC *ansatz* definido como $U(\theta) = U_2(\theta_2) U_1(\theta_1)$, donde ambas U_i implementan compuertas RX parametrizadas y $CNOT$ fijas.

Será de utilidad saber que el estado cuántico inicial $|\phi_0\rangle = |0\rangle^{\otimes n}$ pasa por una transformación S_0 , conocida como una capa de codificación de la entrada, que está parametrizada por la información clásica de entrada $\mathbf{x}$, de modo que codifica la información clásica en los qubits, formando el estado $S_0(\mathbf{x}) |\phi_0\rangle$, conocido como una Aplicación (o Mapa) Cuántica de las *Features* de Entrada (o QIFP por sus siglas en inglés).

3.1.2 Compuertas de capa o capas de procesamiento

En [18], buscando un equivalente al teorema de aproximación universal (UAT por sus siglas en inglés) de las redes neuronales, los autores plantean el esquema (ver figura 3.2) que un clasificador (de un qubit, aunque se puede extender a n qubits) debería seguir para aproximar cualquier función con un grado de precisión arbitrario. Este se debe definir

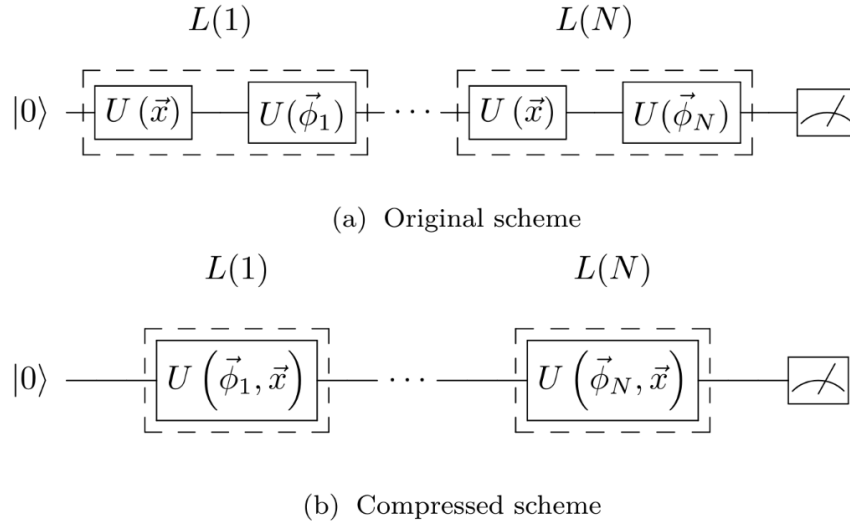


Figura 3.2: Esquemas de un clasificador de un qubit extendido (a) y comprimido o simplificado (b) usando las compuertas de capa [18]. La notación usada puede variar respecto a lo trabajado en este trabajo, pero la idea sigue siendo la misma: componer una unidad de recarga y una VQC *ansatz*.

como $\mathcal{U}(\theta, \mathbf{x}) := U(\theta_k) U(\mathbf{x}) \dots U(\theta_1) U(\mathbf{x})$, y si se definen las compuertas de capa como $L_i(\theta_i, \mathbf{x}) := U(\theta_i) U(\mathbf{x})$, entonces $U(\mathbf{x}) = L_k(\theta_k, \mathbf{x}) \dots L_1(\theta_1, \mathbf{x})$. Lo que se debe resaltar de aquí son dos puntos: en una QNN,

- las $U(\theta_i)$ serán los VQCs ansatz que se trataron anteriormente (3.1.1);
- y $U(x)$ será una unidad de recarga (RUU por sus siglas en inglés) que *incrustará las no-linealidades* dentro del circuito, pues los VQCs serán lineales y unitarios, por lo que este será el equivalente a las funciones de activación en redes neuronales clásicas. Esta puede también ser parametrizada por i , solo que los autores de [18] demostraron que basta con tener la misma en cada capa para cumplir con el UAT.

Nótese que, tal como en las redes neuronales clásicas, $U(x)$ no está parametrizada por ningún elemento de θ (tal como las funciones de activación son, en general, independientes de los pesos y los sesgos).

Dicho esto, se puede proceder a estructurar un QMLP.

3.1.3 Un QMLP estándar

Resumen de la arquitectura del QMLP del artículo

Tal como un perceptrón multicapa clásico se crea componiendo o solapando al menos dos capas de alimentación directa FC (una oculta y otra de salida) [4] [19], una QMLP es la composición de una capa de codificación de la entrada, al menos una compuerta de capa (capa oculta) y una VQC ansatz (capa de salida). Respectivamente, corresponden a S_0 , $L_1(\theta_1, x) = U_1(\theta_1)R_1(x)$ y $U_2(\theta_2)$ en la figura 3.3.

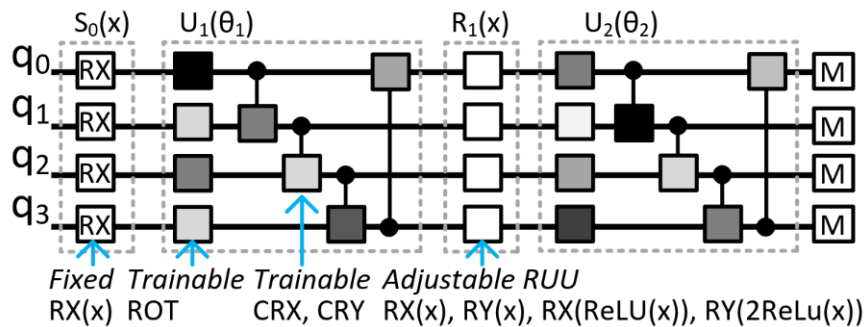


Figura 3.3: La arquitectura canónica de un QMLP de 4 qubits propuesta en [6].

Más precisamente:

- La **capa de codificación de la entrada** se plantea como aplicaciones a cada qubit de una compuerta de rotación RX fija (o traslacionalmente invariante).
- El **VQC ansatz** U_1 (con parámetro(s) θ_1) consta de compuertas Rot parametrizadas aplicadas a cada uno de los qubits (por separado) seguida de un acoplamiento

por vecino más cercano usando compuertas parametrizadas de control de 2-qubits CRX y/o CRY ¹.

- La **RUU** aplica a cada qubit una compuerta de rotación RX o RY , parametrizada por la entrada clásica o la evaluación de alguna aplicación en dicha entrada (como una función de activación clásica).
- El **VQC ansatz** U_2 tiene la misma estructura que U_1 pero con parámetros θ_2 posiblemente distintos.
- La **medición** de los qubits se realiza usando compuertas Z de Pauli.

Aunque no está explícito en la figura 3.3, los valores obtenidos pasan a ser las entradas de una **capa lineal FC** para su posprocesamiento y ‘mapeo’ a la dimensión requerida por el problema de clasificación a tratar (para lo que se ayudan de una función *softmax* logarítmica).

3.1.4 Usada en el artículo

Los autores replican, al pie de la letra, la arquitectura antes descrita, extendiéndola a un circuito de 16 qubits: la codificación angular se hace usando compuertas RX fijas, ambos VQC ansatz se construyen a partir de compuertas Rot y CRX parametrizadas, la RUU usa únicamente compuertas RX parametrizadas y se realizan las mediciones de los qubits usando compuertas Z de Pauli.

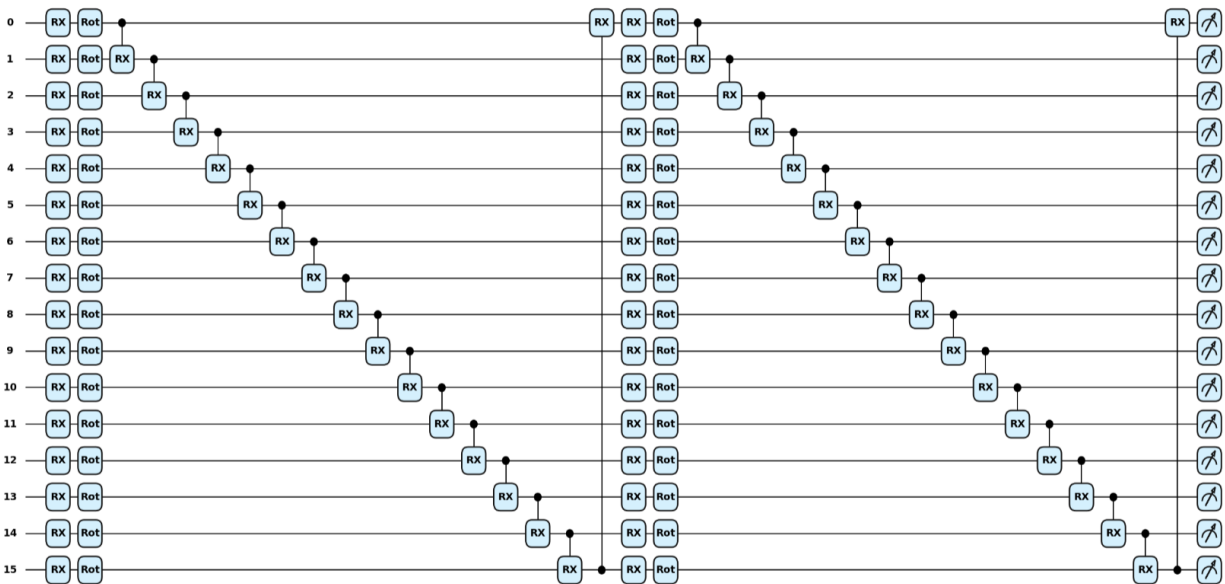


Figura 3.4: Circuito cuántico del QMLP propuesto en el artículo ([13]).

¹La definición de estas compuertas, y de las demás que se utilizarán en las arquitecturas de ambos modelos, se puede ver en detalle en 9.1.

Por otro lado, tal como se puede evidenciar en el primer (de izq. a der.) cuadro del diagrama de flujo provisto en la figura 3.5, la parte clásica no solo consiste en la capa lineal FC de salida que ‘mapea’ las mediciones a un vector de $\mathbb{R}^2$, $\mathbb{R}^4$, $\mathbb{R}^{14}$ o $\mathbb{R}^{23}$, según requiera la tarea de clasificación a trabajar, sino que tiene un componente importante de pre-procesamiento.

Primeramente, se realiza un proceso de normalización estandar usando la función `MinMaxScaler`, de la librería `SciKitLearn`, obteniendo los máximos y mínimos del conjunto de datos de entrenamiento y luego usándolos para normalizar tanto el de entrenamiento como el de evaluación (por lo que puede pasar que una *feature* de alguna entrada particular del conjunto de datos de evaluación no esté en el intervalo $[0, 1]$).

Posteriormente, ya que cada entrada de las bases de datos a trabajar tiene más de 1.500 *features* (como se describió en la sección 1.3) y se trabajarán con solo 16 qubits, se realizó un Análisis de Componentes Principales (PCA por sus siglas en inglés) que permitió extraer un vector de 16 entradas de cada aplicación o PE.

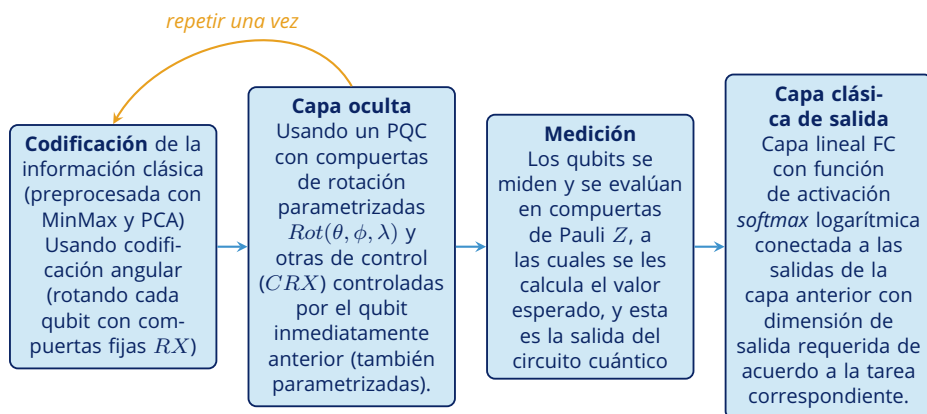


Figura 3.5: Diagrama de flujo del QMLP implementada en el artículo ([13]). En el mismo, aparece un diagrama similar en la figura 2 que se sugiere revisar.

3.2 Implementación

Aunque se tratará más en detalle la implementación de un QMLP y una QCNN en 5, se consideró pertinente mostrar la parte clave de la implementación del QMLP (ver celda 3.1)².

Vale la pena resaltar que el circuito cuántico de la figura 4.2 se implementó usando, en su gran mayoría, la librería `PennyLane` (`qml`), que permite:

- simular el circuito cuántico y conectarlo con la parte clásica (línea 2),
- realizar la codificación angular (líneas 7 y 14),

²Se puede encontrar aquí

- y acceder a las compuertas *Rot*, *RX*, *CRX* y Pauli *Z*, ya sean parametrizadas o no.

Nótese, además, que la capa lineal FC clásica posterior al circuito cuántico (ver figura 4.3) es lo que se retorna al final de la función (línea 29) usando la interfaz de PyTorch que provee PennyLane.

```

1 def make_qmlp_layer(n_qubits=N_QUBITS):
2     qdev = qml.device("default.qubit", wires=n_qubits)
3
4     @qml.qnode(qdev, interface="torch")
5     def qnode(inputs, rot_l1, crx_l1, rot_l2, crx_l2):
6         ##Layer 1 - Data reuploading + trainable Rot + entanglement
7         qml.AngleEmbedding(inputs, wires=range(n_qubits))
8         for i in range(n_qubits):
9             qml.Rot(*rot_l1[i], wires=i)
10        for i in range(n_qubits):
11            qml.CRX(crx_l1[i][0], wires=[i, (i + 1) % n_qubits])
12
13        ##Layer 2 - Data reuploading again + Rot + CRX
14        qml.AngleEmbedding(inputs, wires=range(n_qubits))
15        for i in range(n_qubits):
16            qml.Rot(*rot_l2[i], wires=i)
17        for i in range(n_qubits):
18            qml.CRX(crx_l2[i][0], wires=[i, (i + 1) % n_qubits])
19
20        return [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]
21
22    weight_shapes = {
23        "rot_l1": (n_qubits, 3),
24        "crx_l1": (n_qubits, 1),
25        "rot_l2": (n_qubits, 3),
26        "crx_l2": (n_qubits, 1),
27    }
28
29    return TorchLayer(qnode, weight_shapes)

```

Cuadro 3.1: Celda de código correspondiente a la función `make_qmlp_layer`, en donde se implementa el circuito cuántico del QMLP propuesto en el artículo ([13], se encuentra aquí).

4 QCNN

4.1 Arquitectura

Antes de profundizar en la estructura de las redes neuronales convolucionales cuánticas, se debe mencionar que siguen manteniendo la esencia de una CNN clásica [9]: las entradas pasan por pares de capas convolución-*pooling* y por una capa FC que traduce la entrada transformada en un vector de características de la dimensión deseada. La mayor diferencia estará en cómo se implementan estas capas (código clásico de alto nivel o circuitos cuánticos) y en el pre- y, en ciertos casos, el post- procesamiento que se le debe realizar a los datos antes y después, respectivamente, de pasar por una QCNN.

Por lo anterior, se sugiere al lector, de nuevo, revisar [19] y [4]. Además, vale la pena leer la introducción de la publicación pionera en QCNNs, [9], para entender el paralelo entre el modelo clásico y el cuántico. Dicho esto, se procederá a analizar en detalle las QCNN.

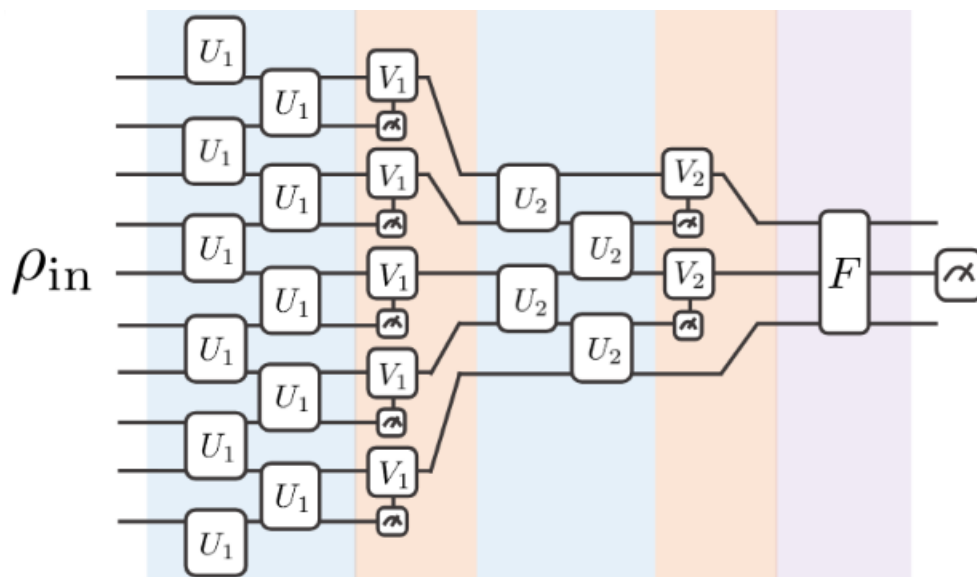


Figura 4.1: Representación gráfica de un circuito correspondiente a una QCNN con dos capas de convolución (azules), dos capas de *pooling* (naranjas), y una FC (morada).

Una QCNN de n qubits es un circuito cuántico parametrizado que recibe un estado

cuántico desconocido, ρ_{in} , y que cuenta con al menos:

- Una capa de **convolución** [9] que,
 - “aplica una única unitaria cuasi-local (U_i) [...]”: quiere decir que U_i se aplica a los qubits y a sus vecinos más próximos. Usualmente, las U_i solo se aplican a 1, 2 o 3 qubits;
 - “[...] de una manera traslacionalmente invariante [...]”: quiere decir que, en una capa de profundidad $k \in \mathbb{N}^+$, se aplicarán k compuertas diferentes;
 - “para una profundidad finita”, donde la profundidad se define así: teniendo una sucesión finita de compuertas cuasi-locales diferentes, $\{U_i\}_{i \in I}$, en la que cada compuerta actúa sobre un subconjunto $S_i \subseteq \{1, \dots, n\}$ de qubits, se define una relación $\prec$ tal que $U_k \prec U_j$ si $S_k \cap S_j \neq \emptyset$ y $k \neq j$. Es decir, una compuerta solo se relaciona con otra si actúan sobre al menos un qubit común.
Con esto en mano, se puede definir el circuito de una capa como un grafo acíclico dirigido [10] $G = (\{U_i\}_{i \in I}, E)$, en el que existe una arista $e = U_k U_j \in E$ solo si $U_k \prec U_j$. De esta manera, se puede definir la profundidad de la capa como el camino (finito) más largo de alguna U_k a alguna U_j .
- Una capa de **pooling** que mide una fracción de los qubits que inicialmente pasaron por ella [9]. Los resultados de esta medición determinan (o controlan) rotaciones unitarias, V_i (cuya cantidad depende de la profundidad de la capa) aplicadas a un (varios) qubit(s) vecino(s).
- Una **FC** [9], que aplica una unitaria F a la fracción de qubits restantes.

Después de que una fracción fija de qubits del estado desconocido pasen por las capas antes mencionadas, se miden y se entregan los valores de las mediciones como salida.

4.1.1 Usada en el artículo

Resumen de la arquitectura de la QCNN del artículo

En la figura 4.2, se puede evidenciar el circuito correspondiente a la parte cuántica del modelo (recordar que es híbrido), propuesto por los autores del artículo, que consiste en una QCNN (**sin ruido**) de $n = 16$ qubits con cuatro capas de convolución de profundidad n y dos capas de *pooling* de profundidad 8 y 4. Por otro lado, en la figura 4.3, se puede evidenciar la parte clásica tanto en el pre- como en el post- procesamiento, en este último, lo clásico consiste en una FC.

Analizando el circuito cuántico de la figura 4.2, lo primero a notar es que los 16 qubits que contienen la información a procesar en el circuito se codifican angularmente usando compuertas de rotación RX , tal como en el QMLP (3.1.4).

Posterior a ello, los qubits pasan por una capa de convolución de profundidad 1 en la que se aplica una misma compuerta parametrizada, $Rot(\theta, \phi, \lambda)$, a cada uno de los qubits.

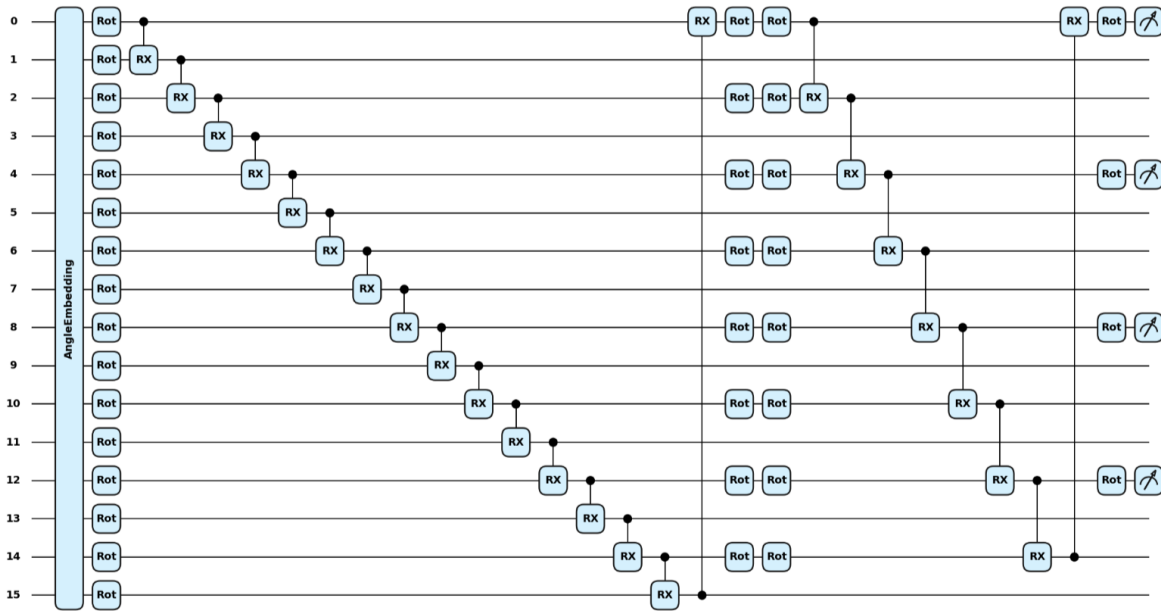


Figura 4.2: Arquitectura del circuito cuántico de la QCNN implementada en [13]

Tras haber pasado por esta, los autores afirman que se introduce entrelazamiento entre los qubits con n compuertas de control RX **parametrizadas**, $CRX(\alpha)$, pero en realidad esto se puede considerar también como una capa de convolución.

Luego de haber sido procesados por las capas de convolución, los qubits entran a una capa de *pooling* sin ruido (ya que los qubits que no se usarán en las capas sucesivas no son medidos y no controlan las compuertas aplicadas a los otros qubits) y de profundidad 1. En esta, se filtran los qubits con índices impares y a los pares se les aplica una compuerta parametrizada Rot . No obstante, algo a notar es que la capa no es invariante traslacionalmente, pues la Rot de cada qubit no-filtrado es distinta a las demás.

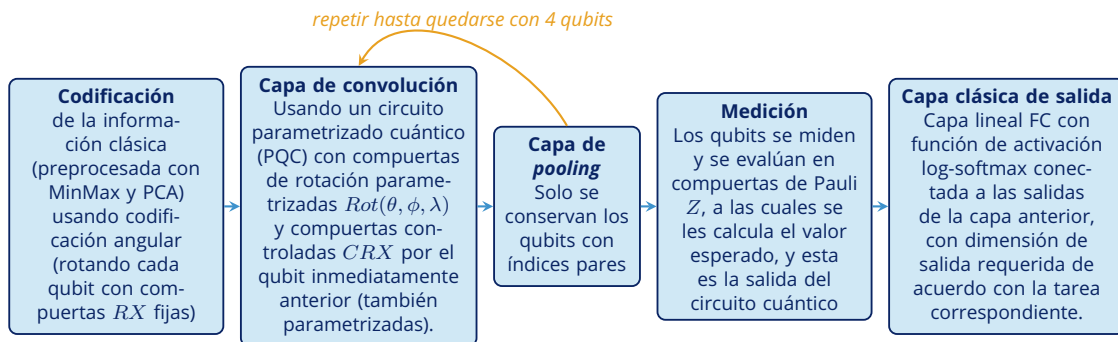


Figura 4.3: Diagrama de flujo de la QCNN implementada en [13]. En la misma referencia, en la figura 3, aparece un esquema similar que se sugiere revisar.

El proceso descrito en los dos anteriores párrafos se repite una vez más, formando

otras dos capas convolucionales y una de *pooling*, y finalmente, se miden los qubits restantes. En suma, aunque no esté descrito en el circuito, la medición de los qubits se realiza aplicándoles una compuerta Pauli Z a cada uno y tomando el valor esperado de esta.

Por otro lado, la parte clásica en el pre- (primer cuadro a la izq. en 4.3) y post- (último cuadro a la der. en 4.3) procesamiento es idéntica a la usada en el QMLP (3.1.4): se realiza una normalización de los datos y un PCA-16 para ajustar la dimensión de la información a los qubits disponibles en el circuito; y se implementa una capa lineal FC para ajustar los valores obtenidos de los cuatro qubits medidos a la dimensión requerida (2, 4, 14 o 23) por la tarea de clasificación que se esté trabajando.

4.2 Implementación

Aunque el código se tratará más en detalle en 5, se consideró pertinente mostrar (celda de código 4.4) la esencia de la implementación de la QCNN antes descrita en Python ¹

Vale la pena resaltar que el circuito cuántico de la figura 4.2 se implementó usando, en su gran mayoría, la librería PennyLane (`qml`), que permite:

- simular el circuito cuántico y conectarlo con la parte clásica (línea 2),
- realizar la codificación angular (línea 6),
- y acceder a las compuertas *Rot*, *CRX* y Pauli Z , ya sean parametrizadas o no.

Nótese, además, que la capa lineal FC clásica posterior al circuito cuántico (ver figura 4.3) es lo que se retorna al final de la función (línea 42) usando la interfaz de PyTorch que provee PennyLane.

¹Se puede encontrar aquí.

```

1 def make_qcnn_layer(n_qubits=N_QUBITS):
2     qdev = qml.device("default.qubit", wires=n_qubits)
3     @qml.qnode(qdev, interface="torch")
4     def qnode(inputs, conv1, entangle1, pool1, conv2, entangle2,
5               pool2):
6         qml.AngleEmbedding(inputs, wires=range(n_qubits))
7         ##Layer 1: conv + CRX entanglement
8         for i in range(n_qubits):
9             qml.Rot(*conv1[i], wires=i)
10        for i in range(n_qubits):
11            qml.CRX(entangle1[i][0], wires=[i, (i + 1) % n_qubits])
12        ##Pooling: keep even qubits
13        qubits_l1 = list(range(0, n_qubits, 2)) # 0, 2, ..., 14
14        for i, w in enumerate(qubits_l1):
15            qml.Rot(*pool1[i], wires=w)
16        ##Layer 2: conv + CRX entanglement on 8 remaining qubits
17        for i, w in enumerate(qubits_l1):
18            qml.Rot(*conv2[i], wires=w)
19        for i in range(len(qubits_l1)):
20            qml.CRX(
21                entangle2[i][0],
22                wires=[qubits_l1[i], qubits_l1[(i + 1) % len(qubits_l1)
23                    ]],
24            )
25        qubits_l2 = list(range(0, len(qubits_l1), 2))
26        final_qubits = [qubits_l1[j] for j in qubits_l2]
27        for i, w in enumerate(final_qubits):
28            qml.Rot(*pool2[i], wires=w)
29
30        return [qml.expval(qml.PauliZ(q)) for q in final_qubits]
31
32    weight_shapes = {
33        "conv1": (n_qubits, 3),
34        "entangle1": (n_qubits, 1),
35        "pool1": (n_qubits // 2, 3),
36
37        "conv2": (n_qubits // 2, 3),
38        "entangle2": (n_qubits // 2, 1),
39        "pool2": (n_qubits // 4, 3)
40    }
41
42    output_dim = n_qubits // 4 #=4
43    return TorchLayer(qnode, weight_shapes), output_dim

```

Figura 4.4: (Celda de código) función `make_qcnn_layer`, que implementa, en Python, de la mano de PyTorch y PennyLane, el núcleo de la QCNN descrita en el artículo y recuperada de aquí.

5 Implementación

5.1 Herramientas usadas

Los autores se ayudaron de herramientas como Conda ¹ para la creación de entornos virtuales y CUDA ² para posibilitar el uso de unidades de procesamiento gráfico (GPUs), pero su principal herramienta fue Python y varias de sus librerías para simular (en un computador clásico) los dos modelos.

Particularmente, Scikit-learn ³ se usó para el preprocesamiento clásico, es decir, la normalización (realizada instanciando de la clase `preprocessing.MinMaxScaler` y usando sus métodos `fit_transform` y `transform`) y PCA-16 (creando una instancia de la clase `decomposition.PCA` y utilizando sus métodos `fit_transform` y `transform`).

Por otra parte, PyTorch ⁴ se usó para el entrenamiento (optimización) del modelo (función de pérdida y funciones de activación) y su inferencia (gracias a la interfaz de PennyLane vista en 4.4 y 3.1), y para posibilitar el uso GPUs.

Específicamente, importando `nn.functional`, se obtuvo acceso a la función de activación *softmax* logarítmica (`log_softmax`) y la función de pérdida de verosimilitud logarítmica negativa (`nll_loss`). Asimismo, fue posible usar la interfaz de PennyLane (de la que se hablará más adelante) gracias a la implementación de una capa lineal en esta librería (`nn.Linear`) y de un módulo de red neuronal (`nn.Module`) que permitió realizar inferencia con los modelos.

Por otro lado, se usó el módulo `optim` de PyTorch para generar una instancia de `optim.Adam`, una clase que implementa el optimizador Estimación Adaptativa de Momentos (Adam), usado en el ciclo de entrenamiento clásico de una red neuronal. Dicho uso se puede evidenciar en la celda 5.1.

Finalmente, PennyLane ⁵, como ya se había mencionado, permitió la construcción del circuito cuántico gracias a la implementación de la codificación angular en la clase `AngleEmbedding`, de las compuertas parametrizadas *Rot* (Rot) y *CRX* (CRX), y la Pauli *Z* (PauliZ), y del valor esperado de una compuerta (`expval`).

¹ Estando a 16 de junio de 2026, se podía encontrar aquí

² Estando a 16 de junio de 2026, se podía encontrar aquí

³ Estando a 26 de mayo de 2026, se podía encontrar aquí

⁴ Estando a 26 de mayo de 2026, se podía encontrar aquí

⁵ Estando a 26 de mayo de 2026, se podía encontrar aquí

```
1     for batch_idx, (inputs, target) in enumerate(train_loader, 0):
2         inputs, target = inputs.to(DEVICE), target.to(DEVICE)
3         optimizer.zero_grad()
4         outputs = model(inputs)
5         loss = F.nll_loss(outputs, target)
6         loss.backward()
7         optimizer.step()
8
9         _, predicted = torch.max(outputs.data, dim=1)
10        total += target.size(0)
11        correct += (predicted == target).sum().item()
12        running_loss += loss.item()
```

Figura 5.1: (Celda de código) fragmento de la función `train`, recuperada el 15 de junio de 2026 de aquí.

Asimismo, tal como se vió en 4.4 y 3.1, usando instancias de `TorchLayer`, se logró ‘in-crustar’ el circuito cuántico dentro de una capa lineal de PyTorch, permitiendo la conexión entre el mundo clásico y el cuántico.

5.2 Modificaciones propuestas

Como se mencionará en la discusión (7), se realizó una *pull request* (que se puede encontrar aquí) en la que se plantearon los siguientes cambios al código fuente usado para simular los QMLPs y las QCNNs:

- Se añadió el archivo `qmlenv_clean.yml`, que permite inicializar el entorno virtual necesario para usar el código en cualquier computador.
- Se añadió `data/ember_sklearn_compat.py` para permitir la extracción de información de EMBER de modo que sea compatible con las librerías utilizadas.
- Se transformó la carpeta `Models` en un paquete local, con el objetivo de agrupar las funciones y clases que `QMLP.py` y `QCNN.py` tienen en común, para hacer al código más legible y reutilizable.
- Se cambió el nombre de algunas carpetas que contienen los modelos de los mejores QMLPs y las mejores QCNNs para asegurar consistencia en ambos casos.

6 Recreación de los experimentos

Después de descargar las bases de datos AZ y EMBER como se describió en la sección 1.3 y hacer las modificaciones al código descritas en la sección 5.2, se tomaron los tres mejores modelos del QMLP y de la QCNN registrados por los autores, y se midió su desempeño (usando las métricas propuestas por ellos) en dichas bases de datos. No obstante, cabe aclarar que para la tarea de clasificación de familias de *malware*, únicamente se evaluaron los modelos con información de las 23 familias más frecuentes.

En la tabla 6.1 y 6.3 se encuentran los resultados de la clasificación binaria en AZ y EMBER, respectivamente. Por otra parte, en las tablas 6.5 y 6.6 se encuentran los resultados de la tarea de clasificación por familias de *malware* en AZ y EMBER, respectivamente.

Cuadro 6.1: Resultados en AZ-Class de los tres mejores modelos de QCNN y de QMLP

Modelo	Exact.	Pérdida	Prec.	Recall	F_1	FPR	FNR	ROC-AUC
qcnn-1	0.0307	4.7435	0.0271	0.0287	0.0279	0.0440	0.9713	0.5169
qcnn-2	0.0219	4.9220	0.0170	0.0205	0.0186	0.0444	0.9795	0.4817
qcnn-3	0.0235	5.2793	0.0192	0.0222	0.0206	0.0444	0.9778	0.4612
qmlp-1	0.0157	6.8691	0.0159	0.0149	0.0154	0.0447	0.9851	0.4616
qmlp-2	0.0161	6.6273	0.0151	0.0151	0.0151	0.0447	0.9849	0.4501
qmlp-3	0.0153	6.6621	0.0144	0.0140	0.0142	0.0447	0.9860	0.4524

Algo a notar es que los resultados en AZ-Class difieren por más de 35 puntos porcentuales a lo reportado en el artículo, como se puede evidenciar en el cuadro 6.4. Similarmente, aunque los resultados reportados y recreados sean más cercanos, en EMBER-Class sigue habiendo una diferencia en el rendimiento reportado y recreado, como se puede observar en 6.2.

En ambos casos, se compararon los resultados de las QCNN recreadas con las reportadas, pues estas fueron las que mejor desempeño obtuvieron en los experimentos. No obstante, en el artículo, los QMLP se desempeñan mucho mejor en esta tarea, llegando a superar a las QCNN por más de 15 puntos porcentuales en exactitud.

Esta divergencia puede deberse a un error en los cambios realizados y descritos en

Cuadro 6.2: Resultados en AZ-Class del mejor modelo recreado y los reportados en el artículo (en puntos porcentuales)

Modelo	Exact.	Prec.	Recall	F_1	FPR	FNR	ROC-AUC
Recreado	3.07	2.71	2.87	2.79	4.40	97.13	51.69
QCNN Rep.	41.7 $\pm$ 1.6	36.1 $\pm$ 0.6	40.5 $\pm$ 1.3	38.19 $\pm$ 0.9	2.7 $\pm$ 0	59.5 $\pm$ 1.3	89 $\pm$ 0.3
$ \Delta $ (media de Rep. Vs. Rec.)	38.63	33.39	37.63	35.40	1.70	37.63	37.31

5, un error de reproducibilidad ¹, un error al descargar y/o manipular las bases de datos -Class (pues para ello se utilizó uno de los archivos nuevos creados como parte de las modificaciones al código 5.2), o alguna otra causa. Se espera que, en futuras discusiones con los autores, se pueda solucionar.

Cuadro 6.3: Resultados en EMBER-Class de los tres mejores modelos de QCNN y de QMLP

Modelo	Exact.	Pérdida	Prec.	Recall	F_1	FPR	FNR	ROC-AUC
qcn-1	0.4237	2.0247	0.2546	0.3110	0.2800	0.0269	0.6890	0.8475
qcn-2	0.5362	1.5403	0.4396	0.3784	0.4067	0.0218	0.6216	0.9110
qcn-3	0.3747	2.1311	0.2124	0.2202	0.2163	0.0294	0.7798	0.8134
qmlp-1	0.4386	2.4119	0.3528	0.3637	0.3582	0.0259	0.6363	0.8736
qmlp-2	0.4731	2.0562	0.3987	0.4007	0.3997	0.0246	0.5993	0.8903
qmlp-3	0.4456	2.4977	0.3544	0.3729	0.3635	0.0255	0.6271	0.8613

En contraste, en las tareas de clasificación binaria, los resultados son mucho más similares: la mayoría de las métricas de los mejores modelos, tanto de QCNN como de QMLP, difieren por menos de un punto porcentual.

No obstante, los resultados de qmlp-3 en EMBER-Domain difieren por alrededor de 10 puntos porcentuales. Esto puede darse si los autores usaron algún otro modelo para reportar los resultados y no es el que está disponible al público. De nuevo, se espera se pueda discutir y resolver esto con los autores en un futuro.

¹Esto se discutió con uno de los autores en GitHub (consultado el 16 de junio de 2026), y puede que se deba a la ausencia de la semilla usada para inicializar el generador de números aleatorios usado.

Cuadro 6.4: Resultados en EMBER-Class del mejor modelo recreado y los reportados en el artículo (en puntos porcentuales)

Modelo	Exact.	Prec.	Recall	F_1	FPR	FNR	ROC-AUC
Recreado	53.62	43.96	37.84	40.67	2.18	62.16	91.10
QCNN Rep.	60.7 ± 0.8	46.7 ± 1.4	49.7 ± 2.8	48.11 ± 1.84	1.83 ± 0	50.3 ± 2.8	92.5 ± 0.3
$ \Delta $ (media de Rep. Vs. Rec.)	7.08	2.74	11.86	7.44	0.35	11.86	1.40

Cuadro 6.5: Resultados en AZ-Domain de los tres mejores modelos de QCNN y de QMLP

Modelo	Exact.	Pérdida	Prec.	Recall	F_1	FPR	ROC-AUC	PR-AUC
qcn-1	0.9160	0.2602	0.7809	0.6958	0.7359	0.3042	0.8537	0.5464
qcn-2	0.9114	0.2655	0.7627	0.6826	0.7204	0.3174	0.8526	0.5246
qcn-3	0.9126	0.2487	0.7661	0.6922	0.7273	0.3078	0.8671	0.5493
qmlp-1	0.9094	0.2647	0.7527	0.6923	0.7212	0.3077	0.8492	0.5129
qmlp-2	0.9143	0.2572	0.7767	0.6825	0.7265	0.3175	0.8653	0.5395
qmlp-3	0.9137	0.2561	0.7701	0.6955	0.7309	0.3045	0.8706	0.5555

Cuadro 6.6: Resultados en EMBER-Domain de los tres mejores modelos de QCNN y de QMLP

Modelo	Exact.	Pérdida	Prec.	Recall	F_1	FPR	ROC-AUC	PR-AUC
qcn-1	0.7472	0.5110	0.7481	0.7433	0.7457	0.2567	0.8171	0.8162
qcn-2	0.7519	0.5154	0.7509	0.7508	0.7508	0.2492	0.8225	0.8191
qcn-3	0.7520	0.5159	0.7514	0.7499	0.7506	0.2501	0.8220	0.8272
qmlp-1	0.7798	0.5122	0.7812	0.7762	0.7787	0.2238	0.8266	0.8234
qmlp-2	0.7171	0.5089	0.7168	0.7177	0.7173	0.2823	0.8259	0.8240
qmlp-3	0.6745	0.5555	0.6758	0.6763	0.6760	0.3237	0.7788	0.7571

7 Discusión

En primer lugar, se debe agradecer a los autores del artículo por haber hecho público el código en el que implementaron ambos modelos y los tres mejores (en teoría) modelos de cada uno ¹, pues esto permite que la comunidad revise con mayor cautela los experimentos que realizaron y continuar con el propósito del artículo: explorar la utilidad de los QMLPs y las QCNNs en la clasificación e identificación de *malware*.

Sin embargo, ciertamente la manera de ejecutarlo no era lo más estructurada ni clara (de ahí que se hicieran los cambios propuestos en esta *pull request*). Esto, en parte, se debe a la ausencia de una documentación clara y extensa donde se explique cómo está organizado el código y cómo se debe utilizar, pues si bien las implementaciones vistas en 3.1 y 4.4 se entienden leyendo el artículo detenidamente y guiándose por las figuras 3.4 y 4.2, comentarios en el resto del código y un archivo `README.md` deberían estar presentes.

En suma, la replicabilidad de los experimentos en su totalidad queda en duda tras los resultados vistos, en especial, en la tarea de clasificación por familias de *malware*. Asimismo, falta la revisión de los cambios realizados y de la correcta funcionalidad de las semillas para inicializar los generadores de números aleatorios (de PyTorch y NumPy) que influyen en la inferencia y, muy probablemente (ya que no se revisó en profundidad esta funcionalidad), en el entrenamiento de los modelos.

Respecto a las simulaciones de los modelos, una comparación interesante se podría dar entre los QMLP y las QCNN de otros números de qubits como 4, 8 y 32, o incluso 1 en el caso de la QMLP, para investigar qué tan bien aproxima la función clasificadora objetivo una arquitectura como la canónica propuesta en [18]. Esto es, además, cuestión de ajustar un hiperparámetro (`n_qubits` en la versión del artículo y `N_QUBITS` en la *pull request*), por lo que es una buena oportunidad para realizar otros experimentos con lo que ya está implementado.

En la misma línea, se podrían analizar ambos modelos con más capas ocultas, en especial la QCNN, que, según los autores, resulta más eficiente de entrenar gracias a la reducción en el número de compuertas usadas por la reducción del número de qubits de la salida por obra de las capas de *pooling* ².

Por otra parte, los autores usaron la implementación de los qubits de PennyLane **sin ruido**: `default.qubit`; y además, no utilizaron compuertas de dos qubits controladas por

¹A 16 de junio de 2026, se podía encontrar en este repositorio.

²En particular, tanto el QMLP como la QCNN tienen un número de compuertas que crece linealmente respecto al número de qubits (N), i.e., su complejidad computacional es $O(N)$ [18] [6] [9].

las mediciones de la mitad de los qubits a eliminar en las capas de *pooling*, tal como originalmente se plantearon en [9]. Esto implica que hace falta evaluar el desempeño de los modelos en un entorno mucho más realista para evidenciar su viabilidad práctica.

Por otro lado, se podrían incorporar en las RUU del QMLP una no-linealidad, componiendo estas unidades con funciones de activación clásicas que ajusten el vector de datos de entrada. Esto con el fin de evidenciar si, añadiendo un elemento clásico dentro del circuito cuántico, se podría mejorar su rendimiento, además de qué tan compatible son las no-linealidades introducidas por las RUU con las de las funciones de activación clásicas.

Finalmente, se podría explorar si realizar el análisis de componentes principales usando un esquema cuántico podría mejorar el desempeño de ambos modelos. Esto es especialmente relevante al entrenar los modelos con la base de datos AZ-Domain, pues el número de registros presentes en ella (tanto de entrenamiento como de evaluación) no difiere por más de 20,000 a un número de registros que permitió una alta generalización en la tarea de detectar intrusos en redes informáticas en [3].

8 Bibliografía

- [1] Kevin Allix et al. "AndroZoo: Collecting Millions of Android Apps for the Research Community". En: *Proceedings of the 13th International Conference on Mining Software Repositories*. MSR '16. New York, NY, USA: Association for Computing Machinery, 2016, págs. 468-471. ISBN: 9781450341868. DOI: 10.1145/2901739.2903508. URL: <https://doi.org/10.1145/2901739.2903508>.
- [2] Hyrum S. Anderson y Phil Roth. *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models*. 2018. DOI: 10.48550/arXiv.1804.04637. arXiv: 1804.04637 [cs.CR]. URL: <https://doi.org/10.48550/arXiv.1804.04637>.
- [3] Armando Bellante et al. *Evaluating the Potential of Quantum Machine Learning in Cybersecurity: A Case-Study on PCA-Based Intrusion Detection Systems*. 2025. DOI: 10.1016/j.cose.2025.104341. arXiv: 2502.11173 [quant-ph]. URL: <https://doi.org/10.1016/j.cose.2025.104341>.
- [4] Andriy Burkov. *The Hundred-Page Language Models Book*. Online book. 2025. URL: <https://www.thebook.com/> (visitado 24-05-2026).
- [5] M. Cerezo et al. "Variational quantum algorithms". En: *Nature Reviews Physics* 3 (2021), págs. 625-644. DOI: 10.1038/s42254-021-00348-9. URL: <https://www.nature.com/articles/s42254-021-00348-9>.
- [6] Cheng Chu et al. "QMLP: An Error-Tolerant Nonlinear Quantum MLP Architecture Using Parameterized Two-Qubit Gates". En: *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*. ISLPED '22. Association for Computing Machinery, 2022, págs. 1-6. DOI: 10.1145/3531437.3539719. URL: <https://doi.org/10.1145/3531437.3539719>.
- [7] Giovanni Ciaramella et al. "Exploring Quantum Machine Learning for Explainable Malware Detection". En: *2023 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2023, págs. 1-6. DOI: 10.1109/IJCNN54540.2023.10191964. URL: <https://doi.org/10.1109/IJCNN54540.2023.10191964>.
- [8] Giovanni Ciaramella et al. "Introducing Quantum Computing in Mobile Malware Detection". En: *Proceedings of the 17th International Conference on Availability, Reliability and Security*. ARES '22. Association for Computing Machinery, 2022, págs. 1-8. DOI: 10.1145/3538969.3543816. URL: <https://doi.org/10.1145/3538969.3543816>.

- [9] Iris Cong, Soonwon Choi y Mikhail D. Lukin. "Quantum Convolutional Neural Networks". En: *Nature Physics* 15 (2019), págs. 1273-1278. DOI: 10.1038/s41567-019-0648-8. arXiv: 1810.03787 [quant-ph]. URL: <https://doi.org/10.1038/s41567-019-0648-8>.
- [10] Reinhard Diestel. *Graph Theory*. 2.^a ed. Vol. 173. Graduate Texts in Mathematics. New York: Springer-Verlag, 2000. ISBN: 0-387-98976-5.
- [11] Matthew G. Gaber, Mohiuddin Ahmed y Helge Janicke. "Malware Detection with Artificial Intelligence: A Systematic Literature Review". En: *ACM Comput. Surv.* 56.6 (ene. de 2024). ISSN: 0360-0300. DOI: 10.1145/3638552. URL: <https://doi.org/10.1145/3638552>.
- [12] Eliška Krátká y Aurél Gábor Gábris. "Quantum Computing Methods for Malware Detection". En: *Machine Learning, Deep Learning and AI for Cybersecurity*. Ed. por Mark Stamp y Martin Jureček. Springer, 2025, págs. 207-228. ISBN: 978-3-031-83157-7. DOI: 10.1007/978-3-031-83157-7_8. URL: https://doi.org/10.1007/978-3-031-83157-7_8.
- [13] Jesus Lopez et al. *Towards Quantum Machine Learning for Malicious Code Analysis*. 2025. DOI: 10.48550/arXiv.2508.19381. arXiv: 2508.19381 [cs.LG]. URL: <https://doi.org/10.48550/arXiv.2508.19381>.
- [14] Francesco Mercaldo et al. "Towards Explainable Quantum Machine Learning for Mobile Malware Detection and Classification". En: *Applied Sciences* 12.23 (2022), pág. 12025. DOI: 10.3390/app122312025. URL: <https://doi.org/10.3390/app122312025>.
- [15] Tuyen Nguyen et al. "An Evaluation of Hardware-Efficient Quantum Neural Networks for Image Data Classification". En: *Electronics* 11.3 (2022). DOI: 10.3390/electronics11030437. URL: <https://doi.org/10.3390/electronics11030437>.
- [16] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge University Press, 2010. ISBN: 9781107002173. DOI: 10.1017/CB09780511976667. URL: <https://doi.org/10.1017/CB09780511976667>.
- [17] Jimin Park et al. "MalCL: Leveraging GAN-Based Generative Replay to Combat Catastrophic Forgetting in Malware Classification". En: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 1. Association for the Advancement of Artificial Intelligence, 2025, págs. 658-666. DOI: 10.1609/aaai.v39i1.32047. URL: <https://doi.org/10.1609/aaai.v39i1.32047>.
- [18] Adrián Pérez-Salinas et al. "Data re-uploading for a universal quantum classifier". En: *Quantum* 4 (2020), pág. 226. DOI: 10.22331/q-2020-02-06-226.
- [19] Simon J. D. Prince. *Understanding Deep Learning*. 2026 online edition. The MIT Press, 2026. ISBN: 9780262048644. URL: <https://udlbook.github.io/udlbook/> (visitado 24-05-2026).
- [20] Tony Quertier y Grégoire Barrué. *Towards an In-depth Detection of Malware Using Distributed QCNN*. 2023. DOI: 10.48550/arXiv.2312.12161. arXiv: 2312.12161 [cs.CR]. URL: <https://doi.org/10.48550/arXiv.2312.12161>.

- [21] Mohammad Saidur Rahman, Scott Coull y Matthew Wright. "On the Limitations of Continual Learning for Malware Classification". En: *Proceedings of The 1st Conference on Lifelong Learning Agents*. Ed. por Sarath Chandar, Razvan Pascanu y Doina Precup. Vol. 199. Proceedings of Machine Learning Research. PMLR, 2022, págs. 564-582. URL: <https://proceedings.mlr.press/v199/rahman22a.html>.
- [22] Mohammad Saidur Rahman et al. "MADAR: Efficient Continual Learning for Malware Analysis with Distribution-Aware Replay". En: *Proceedings of the 2025 Conference on Applied Machine Learning for Information Security*. Ed. por Edward Raff y Ethan M. Rudd. Vol. 299. Proceedings of Machine Learning Research. PMLR, oct. de 2025, págs. 265-291. URL: <https://proceedings.mlr.press/v299/rahman25a.html>.
- [23] Francesc Rodríguez-Díaz et al. "A Survey of Quantum Machine Learning: Foundations, Algorithms, Frameworks, Data and Applications". En: *ACM Comput. Surv.* 58.4 (oct. de 2025). ISSN: 0360-0300. DOI: 10.1145/3764582. URL: <https://doi.org/10.1145/3764582>.
- [24] Soja Rani S y Mouleeswaran. "Image Based Malware Detection Using Transfer Learning and Hybrid Quantum Machine Learning Models". En: *Nanotechnology Perceptions* 20.S6 (2024). DOI: 10.62441/nano-ntp.vi.1527. URL: <https://doi.org/10.62441/nano-ntp.vi.1527>.
- [25] Hanrui Wang et al. "QuantumNAS: Noise-Adaptive Search for Robust Quantum Circuits". En: *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, págs. 692-708. DOI: 10.1109/HPCA53966.2022.00057. URL: <https://doi.org/10.1109/HPCA53966.2022.00057>.
- [26] Hanrui Wang et al. "QuantumNAT: Quantum Noise-Aware Training with Noise Injection, Quantization and Normalization". En: *Proceedings of the 59th ACM/IEEE Design Automation Conference*. DAC '22. Association for Computing Machinery, 2022, págs. 1-6. DOI: 10.1145/3489517.3530400. URL: <https://doi.org/10.1145/3489517.3530400>.
- [27] Ilsun You y Kangbin Yim. "Malware Obfuscation Techniques: A Brief Survey". En: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*. 2010, págs. 297-300. DOI: 10.1109/BWCCA.2010.85.
- [28] Xiaohan Zhang et al. "Enhancing State-of-the-art Classifiers with API Semantics to Detect Evolved Android Malware". En: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. CCS '20. Association for Computing Machinery, 2020, págs. 757-770. DOI: 10.1145/3372297.3417291. URL: <https://doi.org/10.1145/3372297.3417291>.

9 Anexos

Cuadro 9.1: Descripción detallada de las compuertas usadas en los circuitos del QMLP (3.4) y de la QCNN (4.2) planteados en el artículo.

Rotación parametrizada (por θ) alrededor del eje x [16]: $R_X(\theta) = e^{-i\theta X/2}$ $= \begin{pmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$	Rotación parametrizada alrededor de los ejes x (parametrizada por ϕ), y (parametrizada por θ) y z (parametrizada por ω) [16]: $\text{Rot}(\phi, \theta, \omega) = R_Z(\omega)R_Y(\theta)R_X(\phi)$ $= \begin{pmatrix} e^{-i(\phi+\omega)/2} \cos(\theta/2) & -e^{i(\phi-\omega)/2} \sin(\theta/2) \\ e^{-i(\phi-\omega)/2} \sin(\theta/2) & e^{i(\phi+\omega)/2} \cos(\theta/2) \end{pmatrix}$
Rotación alrededor del eje x parametrizada y controlada: $CR_X(\theta) = \begin{pmatrix} I & \mathbf{0} \\ \mathbf{0} & R_X(\theta) \end{pmatrix}$	